

Serie corta – BFS (Python)

Objetivo

Practicar BFS en usos típicos del parcial. Cada ejercicio incluye objetivo, entrada/salida, requisitos y complejidad objetivo. No se incluyen soluciones.

Convenciones sugeridas

- Grafo como lista de adyacencia: `adj: list[list[int]]` (0-index).
- Para grillas: `grid: list[list[int]]` con vecinos 4-dir ($\uparrow \downarrow \leftarrow \rightarrow$).
- Si el grafo puede no ser conexo, indicarlo explícitamente en el enunciado.
- Usar `collections.deque` para la cola.

1) BFS básico: niveles, padres y árbol geodésico

Objetivo: implementar BFS desde s , calcular niveles/distancias mínimas y el árbol BFS (padres).

Entrada: `adj: list[list[int]]`, `s: int`.

Salida: `dist: list[int]` ($\infty = -1$), `parent: list[int]` (-1 en la raíz).

Requisitos:

- Procesar sólo la componente alcanzable desde s .
- Verificar que el árbol BFS es geodésico: `dist[v]` es longitud de $s \rightarrow v$ en G .

Complejidad objetivo: $O(n + m)$.

2) Reconstrucción de camino mínimo (no dirigido o dirigido)

Objetivo: devolver un camino mínimo $s \rightarrow t$ (si existe).

Entrada: `adj: list[list[int]]`, `s: int`, `t: int`.

Salida: `path: list[int]` con los vértices en orden, o `[]` si no hay camino.

Requisitos:

- Usar `parent` de BFS para reconstruir.
- Validar que `len(path)-1 == dist[t]`.

Complejidad objetivo: $O(n + m)$.

3) Bipartitismo por BFS + ciclo impar (opcional)

Objetivo: decidir si un grafo no dirigido es bipartito. Si no lo es, devolver un ciclo impar.

Entrada: adj: list[list[int]]. (El grafo puede tener varias componentes).

Salida:

- Si bipartito: True, color: list[int] con valores {0,1}.
- Si no: False, cycle: list[int] (vértices en orden, sin repetir, cerrado implícitamente).

Requisitos:

- Colorear por niveles (par/impar).
- Si encontrás una arista $u-v$ con $\text{dist}[u] == \text{dist}[v]$, reconstruir ciclo usando parent.

Complejidad objetivo: $O(n + m)$.

4) Componentes por BFS (no dirigido)

Objetivo: listar todas las componentes conexas y su tamaño usando BFS múltiple.

Entrada: adj: list[list[int]].

Salida: comps: list[list[int]] (una lista por componente, vértices en orden de descubrimiento).

Requisitos:

- No repetir visitas; recorrer todas las componentes.
- Ordenar comps por el mínimo índice de cada componente.

Complejidad objetivo: $O(n + m)$.

5) Distancias mínimas en dirigido desde origen múltiple

Objetivo: dado un conjunto de fuentes S , calcular $\text{dist}[v] = \min_{s \in S} \text{dist}_G(s, v)$.

Entrada: adj: list[list[int]] (dirigido), sources: list[int].

Salida: dist: list[int] ($\infty = -1$).

Requisitos:

- Inicializar la cola con todas las fuentes (distancia 0).
- No usar varias BFS separadas.

Complejidad objetivo: $O(n + m)$.

6) BFS en grilla con obstáculos

Objetivo: en una grilla binaria (0 = libre, 1 = bloqueado), hallar la cantidad mínima de pasos entre s y t moviéndose $\uparrow \downarrow \leftarrow \rightarrow$.

Entrada:

- grid: list[list[int]] de tamaño $m \times n$,

- s: tuple[int,int], t: tuple[int,int].

Salida: dist: int (pasos), o -1 si es imposible.

Requisitos:

- Chequear límites y celdas bloqueadas.

- Pista: visited[m][n] y deque.

Complejidad objetivo: $O(m \cdot n)$.

7) Multi-source BFS en grilla (expansión simultánea)

Objetivo: dado un subconjunto de celdas fuente (k orígenes), calcular para cada celda libre la distancia mínima a alguna fuente.

Entrada: grid: list[list[int]] (0/1), sources: list[tuple[int,int]].

Salida: dist: list[list[int]] (-1 en inalcanzables o bloqueadas).

Requisitos:

- Encolar todas las fuentes con distancia 0.

- Expansión en capas.

Complejidad objetivo: $O(m \cdot n)$.

8) BFS en espacio de estados (sumas mod k) — variante del ejercicio 8 de la guía

Objetivo: encontrar la mínima cantidad de movimientos para transformar v1 en w moviéndose en la grilla y sumando el valor de la celda destino módulo k.

Entrada:

- grid: list[list[int]] con valores en $0..k-1$,

- k: int, start: tuple[int,int], v1: int, w: int.

Salida: min_moves: int o -1 si no se puede.

Requisitos:

- Modelar estados como (i, j, r) donde r es el residuo actual modulo k.

- Estado inicial: (start_i, start_j, v1 % k); objetivo: cualquier (i,j,w%k).

Complejidad objetivo: $O(k \cdot m \cdot n)$.

9) BFS para orden topológico con Kahn (dirigido acíclico)

Objetivo: implementar Kahn (BFS sobre indegrados) y detectar si hay ciclo.

Entrada: adj: list[list[int]] (dirigido).

Salida:

- Si es DAG: True, order: list[int] (orden topológico).
- Si no: False, [].

Requisitos:

- Calcular indeg[v]. Encolar todos los v con indeg[v]==0.
- Procesar y reducir indegrados; si al final no procesaste n vértices, hay ciclo.

Complejidad objetivo: $O(n + m)$.

10) Distancias por capas y excentricidad del nodo

Objetivo: dado s, calcular su excentricidad $\text{ecc}(s) = \max(\text{dist}(s,v))$ en su componente.

Entrada: adj: list[list[int]], s: int.

Salida: ecc: int y layers: list[list[int]] (los vértices por capa 0,1,2, ...).

Requisitos:

- Construir layers durante BFS (apilando por distancia).
- Ignorar vértices inalcanzables (otra componente).

Complejidad objetivo: $O(n + m)$.

Sugerencias de firmas

- bfs_tree(adj: list[list[int]], s: int) -> tuple[list[int], list[int]]
- shortest_path(adj: list[list[int]], s: int, t: int) -> list[int]
- bipartite_bfs(adj: list[list[int]]) -> tuple[bool, list[int] | list[int]]
- connected_components_bfs(adj: list[list[int]]) -> list[list[int]]
- multisource_dist_directed(adj: list[list[int]], sources: list[int]) -> list[int]
- grid_shortest_path(grid: list[list[int]], s: tuple[int,int], t: tuple[int,int]) -> int
- grid_multisource_dist(grid: list[list[int]], sources: list[tuple[int,int]]) -> list[list[int]]
- min_moves_modk(grid: list[list[int]], k: int, start: tuple[int,int], v1: int, w: int) -> int
- topo_kahn(adj: list[list[int]]) -> tuple[bool, list[int]]
- eccentricity_layers(adj: list[list[int]], s: int) -> tuple[int, list[list[int]]]